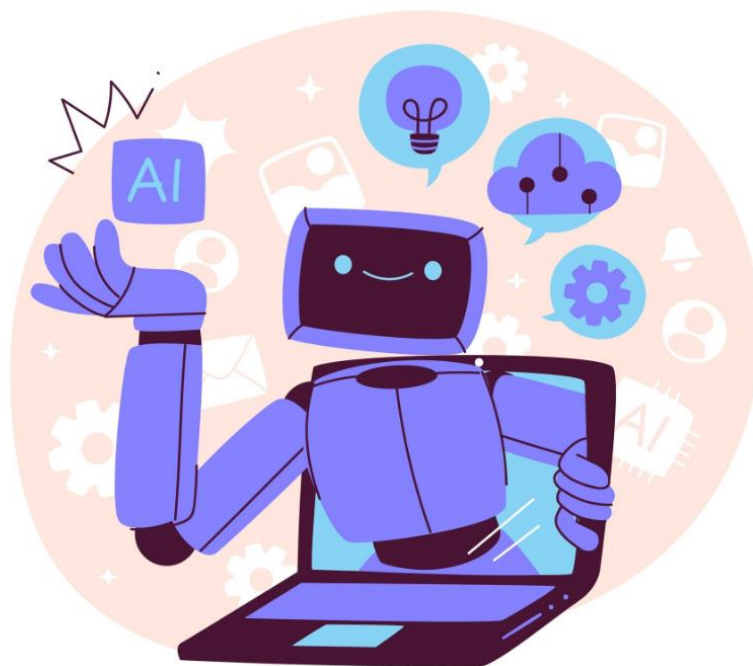


Projet – Robot



Membres :

Amel Ben Chabane

Melia Tafat

Diana Araour

Wissam Arkam

Tassadit Aksas

Sommaire :

1. Introduction.
2. Structure générale du package
 - 2.1.Module Model
 - 2.1.1. `__init__.py`
 - 2.1.2. `Robot.py`
 - 2.1.3. `Arene.py`
 - 2.1.4. `Adapter.py`
 - 2.2.Module Controller
 - 2.2.1. `__init__.py`
 - 2.2.2. `Controller.py`
 - 2.3.Module View
 - 2.3.1. `__init__.py`
 - 2.3.2. `View_2d.py`
 - 2.3.3. `View_3d.py`
 - 2.4. `Main.py`
3. Simulation et environnement
4. Conclusion

1. Introduction

Ce projet a été réalisé dans le cadre de l'UE LU2IN013 – Projet de Développement à Sorbonne Université.

L'objectif était de concevoir une architecture logicielle complète pour simuler un robot mobile de type GoPiGo3, capable de se déplacer dans une arène, d'éviter les obstacles, et de suivre différentes stratégies de contrôle.

Le projet a été développé en Python, avec une architecture MVC, l'utilisation de plusieurs design patterns, et une visualisation 2D ou 3D du robot.

Toutes les sources sont disponibles sur le GitHub public du projet :

<https://github.com/Projet-de-developpement-75005/Projet-Robot.git> (branche 'project').

2. Structure générale du package :

```
C:\USERS\AKSTI\PROJET-ROBOT
├── .gitignore
├── main.py
├── controller
│   ├── controller.py
│   ├── __init__.py
│   └── __pycache__
│       ├── controller.cpython-313.pyc
│       └── __init__.cpython-313.pyc
├── Model
│   ├── adapter.py
│   ├── arene.py
│   ├── robot.py
│   ├── __init__.py
│   └── __pycache__
│       ├── adapter.cpython-313.pyc
│       ├── arene.cpython-313.pyc
│       ├── obstacle.cpython-313.pyc
│       ├── robot.cpython-313.pyc
│       └── __init__.cpython-313.pyc
└── view
    ├── view_2d.py
    ├── view_3d.py
    ├── __init__.py
    └── __pycache__
        ├── view_2d.cpython-313.pyc
        ├── view_3d.cpython-313.pyc
        └── __init__.cpython-313.pyc
```

Les fichiers `__init__.py` servent à regrouper et rendre accessibles les composants principaux de chaque module, cela permet un import centralisé et facilite l'utilisation des modules dans le reste du projet.

2.1 Module Model :

- **Robot.py :**

Ce fichier contient la classe Robot, et est fondamental pour modéliser le mouvement du robot dans l'arène.

Le constructeur `__init__` :

Initialise la position (x, y) du robot.

Initialise son orientation (angle en radians)

Initialise la vitesse de ses roues gauches et droite, la taille des roues et l'écart entre elles.

Et calcul un rayon d'action, et initialise la distance parcourue à zéro.

Méthodes :

-`set_vitesses(self, new_vitesse_g, new_vitesse_d)` : met à jour les vitesses des roues et affiche un message .

-`get_distance()` : renvoie la distance totale parcourue

-`deplacement(dt)` : simule l'avancement du robot pendant un intervalle dt, calcul la nouvelle position à partir de la vitesse moyenne et de l'angle d'orientation, et met à jour la distance totale parcourue.

-`rotation(dt)` : simule la rotation du robot en cas de vitesses différentes entre les roues, calcule un changement d'angle à partir de la différence de vitesse, et normalise l'angle pour qu'il reste dans $[0, 2\pi]$.

- **Arene.py :**

Ce fichier contient deux classes essentielles pour la modélisation de l'environnement, obstacle et arene.

-Classe Obstacle :

Représente un obstacle rectangulaire positionné à (x, y) avec une largeur et une longueur.

Collision (rx, ry, rayon) : calcule si un point (rx, ry) à distance rayon intersecte l'obstacle. Utilise la technique de projection pour tester l'intersection avec une forme circulaire (le robot).

-Classe Arene :

Représente l'environnement de simulation avec une largeur, hauteur, et une liste d'obstacles.

Placer_robot(robot) : enregistre le robot actif dans l'arène

Ajout_obstacles(obstacle) : ajoute un obstacle à la liste

Gerer_collisions(robot) : méthode de détection et de réaction aux collisions.

Elle teste, pour chaque obstacle, si la position actuelle du robot est en intersection avec un obstacle en tenant compte de son rayon.

En cas de collision détectée, le robot est immédiatement stoppé (set_vitesses(0, 0)) pour éviter un chevauchement prolongé.

- Adapter.py :

Ce fichier contient la classe AdapterVirtuel, qui joue le rôle d'adaptateur entre le modèle (robot) et la Controller, et est essentiel pour ne pas soucier de la nature du robot (virtuel ou réel).

Le constructeur :

Reçoit une liste d'obstacles et un robot, et récupère des paramètres comme l'écart entre les roues et le rayon.

Méthodes :

definir_vitesse(self, v_gauche, v_droite) : met à jour les vitesses des roues et le temps de dernière mise à jour.

get_dt(self) : calcule le temps écoulé depuis la dernière mise à jour.

update(self) : met simplement à jour le timestamp.

avancer(self) : appelle déplacement(dt) du robot avec le bon pas de temps.

tourner(self) : appelle rotation(dt) du robot.

dist_parcourue(self) : renvoie la distance parcourue par le robot.

get_ang_parcourue(self) : calcule l'angle tourné en degrés depuis le début à partir de la différence de vitesse entre les roues et du temps.

2.2 Module Controller :

- Controller.py

Ce fichier contient le contrôleur principal de la simulation ainsi que les différentes stratégies de déplacement implémentées.

-Classe Controller :

-Son constructeur prend en paramètre un adaptateur (virtuel ou réel) et une arène.

-run_simulation(strategie, view=None): boucle principale qui :
lance la stratégie (via start)
exécute update () jusqu'à la fin de la stratégie
met à jour le robot et les collisions
met à jour la vue graphique si présente

-Stratégies :

StrategieAvance(distance_cible, vitesse) : fait avancer le robot tout droit sur une distance cible. Lors du start, elle mémorise la distance actuelle. Le update vérifie si la distance parcourue depuis le début a atteint la cible.

StrategieTourner(angle_deg, vitesse_rotation) : fait tourner le robot d'un angle donné à une vitesse angulaire. Lors du start, elle calcule l'angle final cible. update surveille la différence entre l'angle courant et la cible, et arrête la rotation si la cible est atteinte.

StrategieConditionnelle(condition_fonction, action1, action2) : permet de choisir dynamiquement une stratégie selon une condition booléenne. Si la fonction de condition est vraie, action1 est choisie, sinon action2. Elle permet de réagir aux états du robot (par exemple distance, capteur).

StrategieSequentielle(liste_etapes) : permet d'enchaîner plusieurs stratégies dans un ordre défini. Lorsqu'une stratégie est terminée (update retourne True), la suivante est lancée automatiquement.

Chaque stratégie respecte l'interface suivante :

Start(adapter) : initialisation de l'état de la stratégie.

Update(adapter) : progression de l'action, retourne True si elle est terminée.

Stop(adapter) : arrêt explicite.

2.3 Module View

- View_2d.py

Il fournit une visualisation 2D de l'environnement et du robot à l'aide de la bibliothèque tkinter. Il contient une seule classe View

Le robot est représenté comme un rectangle dont l'orientation dépend de l'angle courant (angle_orientation).

Chaque roue est affichée comme un cercle positionné à gauche et à droite du centre.

La méthode update_affichage() est appelée à chaque étape pour rafraîchir l'affichage complet.

La fenêtre est créée avec un Canvas de dimensions adaptées à l'arène.

Les obstacles sont dessinés sous forme de rectangles fixes.

Le robot est modélisé comme un polygone dont la position et l'orientation varient à chaque `update_affichage()`.

La forme du robot dépend de son écart de roues et de la taille de ses roues.

La méthode utilise des transformations trigonométriques (`cos`, `sin`) pour calculer les coordonnées globales des coins du robot à partir de son orientation.

La trajectoire est conservée dans `trace_points`, une liste mise à jour à chaque affichage.

La méthode `create_line()` relie les anciens points de passage du robot pour tracer son chemin.

Les roues sont dessinées comme deux cercles placés de part et d'autre de l'axe principal du robot.

- **View_3d.py**

Le fichier `view_3d.py` propose une visualisation 3D immersive du robot et de son environnement à l'aide de la bibliothèque Ursina, qui exploite les fonctionnalités d'un moteur 3D léger basé sur Python.

Fonctionnalités principales :

Création d'un sol vert simulant l'arène à l'aide d'un plan texturé.

Modélisation du robot comme un parallélépipède bleu (Entity), dont la position et l'orientation sont mises à jour à chaque appel de `update_affichage()`.

Affichage de tous les obstacles présents dans l'arène sous forme de blocs roses positionnés selon leurs dimensions.

Caméra libre (EditorCamera) permettant une navigation fluide en hauteur.

Ajout d'un label affichant les informations en temps réel : position du robot et vitesses des roues.

Gestion du clavier avec une entité `Input_handler`, permettant notamment de quitter la simulation avec la touche Échap.

Détail technique :

Le robot est affiché avec une orientation corrigée (`rotationY`) pour correspondre au repère d'Ursina (sens horaire).

Les entités graphiques sont créées à partir de modèles 3D standards (`cube`, `plane`).

La méthode `step()` de l'application est appelée manuellement pour mettre à jour la scène à chaque itération.

3. Simulation et environnement :

Le système de simulation repose sur l'interaction entre le robot, l'arène et les stratégies appliquées par le contrôleur. L'objectif est de reproduire au mieux les comportements physiques d'un robot réel dans un espace délimité.

- L'environnement est défini par l'objet Arene, qui contient une surface de déplacement ainsi que des obstacles statiques.
- Le robot se déplace dans cet espace et peut entrer en collision avec les obstacles. Chaque collision est gérée automatiquement via la méthode `gerer_collisions()` qui stoppe le robot en cas de contact.
- Le contrôleur applique une ou plusieurs stratégies (séquentielles, conditionnelles) et met à jour le robot à chaque itération de simulation.
- Le temps est simulé à l'aide de `time.sleep()` et `get_dt()`, permettant une synchronisation des mouvements du robot avec une boucle de mise à jour réaliste.
- L'utilisateur peut visualiser les mouvements du robot dans un affichage 2D ou 3D. En 2D, le canvas tkinter affiche en temps réel la trajectoire et la position du robot.

4. Conclusion :

Ce projet nous a permis de développer un simulateur de robot mobile à partir d'une architecture logicielle claire, modulaire et évolutive. Grâce à l'utilisation des patrons de conception et à une séparation stricte des responsabilités (MVC), nous avons pu expérimenter différents comportements de déplacement tout en assurant la maintenabilité du code.

La simulation 2D s'est révélée efficace pour tester les trajectoires et stratégies. La détection de collision est opérationnelle, et les stratégies comme le tracé d'un carré sont fonctionnelles.